KLE Technological University | Creating Value, Leveraging Knowledge

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Capstone Project Report

on

Kedo - AI-Powered Nutrition Assistant

Submitted

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted By

Ansh Ritesh Savadatti	01FE22BCS233
Kadappa Savalagi	01FE22BCS181
Aradhaya Ajit Gaonkar	01FE22BCI019
Yukta Jain	01FE22BCS062

Under the guidance of

Prof. Vijay Biradar
Assistant Professor

Department of Computer Science & Engineering
KLE Technological University, Hubballi

2025–2026

KLE Technological University

Creating Value, Leveraging Knowledge

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

2025-26

CERTIFICATE

This is to certify that project entitled "Kedo - AI-Powered Nutrition Assistant" is a bonafied work carried out by the student team Ansh Ritesh Savadatti (01FE22BCS233), Kadappa Savalagi (01FE22BCS181), Aradhaya Ajit Gaonkar (01FE22BCI019), and Yukta Jain (01FE22BCS062), in partial fulfillment of the completion of 8^{th} semester B. E. course during the year 2025– 2026. The project report has been approved as it satisfies the academic requirement with respect to the project work prescribed for the above said course.

Prof. Vijay Biradar
(Assistant Professor)

Head, Department of CSE
Dr. Vijayalakshmi M.

External Viva-Voce

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

ABSTRACT

The rise in lifestyle-related health problems around the world, like diabetes and heart disease, shows that we need better and more adaptive ways to manage what we eat. Older diet apps don't work very well because they usually just give the same generic advice to everyone, without adjusting to a person's changing metabolism or specific biological needs. Also, while new AI-based health assistants are popular, they have a big flaw: they sometimes "hallucinate" and make up facts. This can be extremely dangerous if someone is relying on the app to avoid allergies or manage a medical condition.

To help solve these problems, this project introduces **KEDO**, an AI-Powered Personalized Nutrition Assistant. KEDO is built using Large Language Models, specifically Gemini, to provide smart dietary advice. Instead of using basic rules, KEDO uses a dynamic approach that actively double-checks its own work.

To make sure the advice is safe and accurate, KEDO uses a strict validation system built with Pydantic. This forces the AI to follow the user's specific health requirements and allergy restrictions before any recipe is shown on the screen. This structured method helps prevent AI mistakes while still creating personalized calorie and macro recommendations based on the user's exact profile.

We built KEDO using a modern tech stack. We used React Native for a smooth mobile app experience and FastAPI in Python for a fast backend. The result is a reliable and smart nutrition tool that gives safe food advice. By helping users manage their health better, KEDO directly supports the United Nations Sustainable Development Goal (SDG) 3.4, which aims to promote preventive healthcare and reduce lifestyle diseases.

Keywords : *Personalized Nutrition, Artificial Intelligence, AI Safety, React Native, FastAPI, Dietary Management.*

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without mentioning the individuals whose professional guidance, encouragement, and support helped us in the successful completion of this report work.

We take this opportunity to express our sincere gratitude to Dr. Ashok Shettar, Pro Chancellor, Dr. Prakash Tewari, Vice Chancellor, and Dr. Basavaraj Anami, Registrar, KLE Technological University, Hubballi, for providing us with the opportunity and facilities to carry out this project successfully.

We would also like to thank Dr. Meena S. M., Dean, Faculty of Computer Science and Engineering, and Dr. Vijayalakshmi M., Head, Department of Computer Science and Engineering, for providing an excellent academic environment that nurtured our practical skills and contributed to the success of our project.

We sincerely thank Dr. P. G. Sunitha Hiremath, Professor, Department of Computer Science and Engineering, for her constant support, inspiration, and wholehearted cooperation as Project Coordinator throughout the course of this work.

We express our heartfelt gratitude to our guide, Prof. Vijay Biradar, Assistant Professor, Department of Computer Science and Engineering, for the valuable guidance, encouragement, and continuous support extended during the completion of the project.

Our gratitude will not be complete without thanking our seniors who have been a constant support and inspiration.

Ansh Ritesh Savadatti - 01FE22BCS233
Kadappa Savalagi - 01FE22BCS181
Aradhaya Ajit Gaonkar - 01FE22BCI019
Yukta Jain - 01FE22BCS062

CONTENTS

LIST OF FIGURES

Chapter 1

INTRODUCTION

1.1 Motivation

The main reason we decided to build KEDO comes from a few major problems we noticed in how people handle their diet and nutrition today:

- **Health Issues on the Rise:** Lifestyle-related diseases, like obesity and type-2 diabetes, are becoming much more common everywhere. A lot of these problems are tied directly to bad dietary habits, making food a really important part of preventive care.
- **Basic and Rigid Apps:** Most of the diet apps you find on the market right now give very generic advice. They mostly just act as basic calorie counters instead of being smart assistants, and they don't really adapt to what a specific person actually needs.
- **AI Mistakes:** Even though Large Language Models (LLMs) are great at generating text, using them for health advice can be risky. Standard AI models sometimes make things up (called hallucinations). For example, suggesting a peanut butter recipe to someone with a nut allergy could be really dangerous.
- **Need for Reliable Tools:** We realized there is a real need for a health tool that relies on validated facts rather than just guessing.

1.2 Problem Statement

"To build a reliable AI-powered nutrition app that gives personalized food advice while preventing dangerous AI mistakes, ensuring safety through data checks and a fallback mechanism."

1.3 Core Objectives

To solve the problems mentioned above, we set a few main goals for the KEDO system:

1. **Better Prompt Design:** We want to build an AI engine that actually understands a user's specific tastes, allergies, and health goals by giving the AI very strict instructions.
2. **Dynamic Recommendations:** Create a backend process that feeds the user's current situation directly into the AI, so recipe suggestions actually match the food they have available in their pantry.
3. **Checking the Output:** Set up a system using strict data schemas to double-check the AI's response before the user ever sees it, just to be safe.
4. **Easy to Use:** Build a simple, mobile-first app that hides all the complicated backend stuff and just gives the user a clean, easy-to-read nutrition dashboard.

Chapter 2

LITERATURE SURVEY

To build KEDO, we looked at existing research and the apps currently available on the market. This helped us see what works well and, more importantly, what is missing from current nutrition tools.

2.1 Standard Diet Apps: Content-Based Filtering

Most of the older food recommendation systems use simple filtering methods. As pointed out in recent research on food apps [1], these systems match basic food details like calories or ingredients to what the user says they like. They usually read recipes and find similarities using standard algorithms.

Limitations: While these systems are okay for simple searches, they are very rigid. They have a hard time adapting if a person's diet changes suddenly or if they need specific real-time advice.

2.2 Generative AI and RAG

Using Retrieval-Augmented Generation (RAG) [2] helped make AI models more factual. New systems like SORT-AI [3] try to make AI safer by checking its work. They act like a safety net, analyzing how the AI builds its answers to make sure it doesn't ignore the rules it was given.

Limitations: Even with these checks, standard setups can still make big mistakes (hallucinations) if the rules aren't set up specifically for strict nutritional data.

2.3 Smart Wearables and Health Predictions

Recent work on adding AI to health wearables [4] shows a shift toward active monitoring. These tools can give early warnings for things like heart or metabolic issues by connecting sensor data to cloud AI. They use algorithms to spot health risks automatically without needing the user to type everything in.

Limitations: They are great at telling you when something is wrong, but they usually can't tell you exactly what meal to eat to help fix the problem.

2.4 Validation Models: MEGA-RAG

The MEGA-RAG framework [5] is a newer way to stop AI mistakes in public health advice. It double-checks facts against multiple sources before showing the final answer to the user.

Limitations: It has good structure, but it lacks a strong fallback option if the AI is just too confused to give a safe answer. This is something we wanted to fix in KEDO.

2.5 Identified Research Gaps

After reviewing the literature, we found two main areas where existing apps fall short:

- **Missing Fallback Plan:** None of the systems we looked at have a good way to switch to a web search or say "I don't know" when they aren't confident about a rare medical condition or ingredient.
- **Lack of a Double-Check System:** Consumer apps usually don't have a secondary check to verify calories and allergens right before the user sees the output.

Chapter 3

SYSTEM ARCHITECTURE

The architecture of KEDO is designed to keep different parts of the app separate, which helps it run faster and keeps user data safe. The system is broken down into a few connected layers.

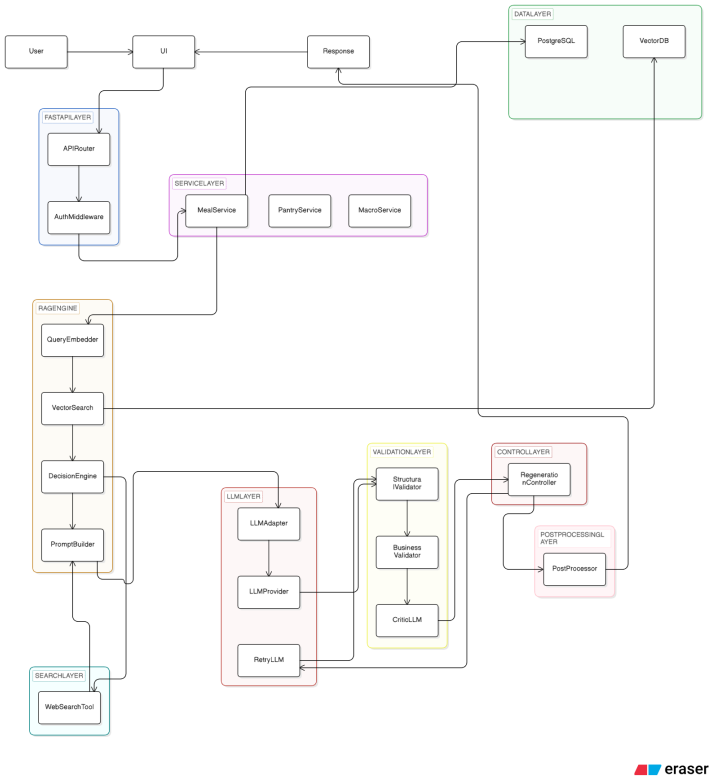

Figure 3.1: KEDO High-Level System Architecture and Agent Workflow

3.1 Architectural Layers

The KEDO platform uses a standard multi-layered approach:

- **The API Layer:** Built with FastAPI, this layer acts as a bridge between the frontend app and the AI engine. It handles simple authentication using the user's email ID, routes requests, and structures the data.

- **The Generation Engine:** This is where the AI processing happens using models like Gemini. It reads the user's details and generates dietary suggestions based on the prompts we give it.
- **The Validation Layer:** Before any response is sent back to the app, it gets checked against JSON schemas using Pydantic. This makes sure the output format is correct and reliable.

3.2 Core Logic and Data Management

KEDO uses standard techniques to handle user inputs effectively:

- **Prompt Engineering:** We use custom code to build strict instructions for the AI, tailoring them to each user's health profile.
- **Data Checks:** We monitor the AI's output to make sure it follows the rules we set, like sticking to a maximum calorie limit.

To keep the system stable, we rely on:

- **JSON Schemas:** We use Pydantic to make sure the AI always returns data in the exact format the frontend expects, which prevents app crashes.
- **State Management:** We hold the user's profile and current pantry inventory in memory so the AI always has the right context.

3.3 The Validation Process

Our setup uses a straightforward way to try and stop the AI from hallucinating:

1. **Formatting Checks (Pydantic):** AI models sometimes mess up the formatting. This layer forces the AI to output proper JSON. If the AI messes up, the system catches it before it breaks the app.

2. **Prompt Limits:** Instead of using complicated multi-agent setups, KEDO just acts as a wrapper around the Gemini API. By feeding the user's specific data directly into the prompt, we guide the AI to verify calories and check for allergens right away.

3.4 Pantry Engine and Note Uploads

A very helpful feature of KEDO is the "Current Pantry" module. This state-management system keeps track of the food items the user currently has at home. Instead of suggesting recipes with missing ingredients, KEDO lets users type in what they have (for example, "I have 2 eggs, some spinach, and some bread"). The AI formats this information and stores it in the system.

Later, the generation engine can be set to a "Pantry-Only" mode. When this is on, the AI is told to only suggest meals using those available items, which helps cut down on food waste. We also added a feature where users can upload doctors' notes, so KEDO can add clinical restrictions to their profile.

Chapter 4

TECHNOLOGIES USED

To build KEDO, we selected a mix of modern and reliable technologies. The tech stack covers the mobile app frontend, the backend API, the AI integration, and how we handle data.

4.1 Frontend and Mobile Application

The user interface is the part of KEDO that people actually interact with, so it needed to be smooth and easy to use.

- **React Native & Expo:** KEDO is built with React Native [6], which lets us write code once and run it on both iOS and Android. We used Expo to speed up the development process, as it makes testing on real devices and handling permissions much easier.
- **JavaScript:** The frontend is written in standard JavaScript. It handles the logic for the app and processes the data that comes back from the AI backend.
- **React Navigation:** We use this library to handle moving between different screens in the app, like going from the dashboard to the recipe details.
- **Axios:** A tool we use to make network requests from the mobile app to our Python backend server.

4.2 Backend and API

The backend is responsible for talking to the AI and managing the data without slowing down the app.

- **Python 3.x:** Python is the main language for our backend. It is the standard language for AI projects and has great libraries for what we needed.

- **FastAPI:** We chose FastAPI [7] because it is very fast and easy to work with. It handles asynchronous requests, which means it can wait for the AI to respond without blocking other users.
- **Uvicorn:** This is the server that actually runs our FastAPI application.
- **Pydantic:** We use Pydantic alongside FastAPI to check that all the data coming in and going out is formatted correctly.

4.3 Artificial Intelligence

The AI is what makes KEDO smart, acting as the brain behind the dietary advice.

- **Large Language Models (Gemini):** We use Google's Gemini models to understand the user's input and generate recipes. These models have enough general knowledge to handle nutrition advice.
- **Prompt Engineering:** Instead of building a really complicated AI system, we kept things simple. The backend takes the user's profile and directly pastes it into the prompt we send to Gemini. This gives the AI all the context it needs to generate a good response.

4.4 Data Storage

We needed a way to store user information and pantry items while the app is running.

- **In-Memory State Management:** Currently, KEDO uses an in-memory STATE to store data. This means that user profiles, pantry items, and session details are kept in the server's memory. Because of this, the data resets whenever the backend restarts. While this works well for testing and fast prototyping, a permanent database might be added later.

4.5 Development Tools

We also used a few standard tools to help us build and test the project.

- **Git and GitHub:** We used these to keep track of our code changes and work together on the project.
- **Postman:** We used Postman to test our backend API endpoints before we connected them to the React Native app.

Chapter 5

SYSTEM REQUIREMENTS AND GOALS

5.1 Hardware and Software Specifications

To run and develop the KEDO app, you need a computer that meets certain basic requirements, mainly to handle the AI development and app testing.

Hardware Requirements (Development Machine):

- **Processor:** Intel i5 / AMD Ryzen 5 or higher is recommended so things run smoothly.
- **RAM:** At least 8 GB, but 16 GB is better if you have a lot of tools running at the same time.
- **Storage:** About 20 to 50 GB of free space. An SSD makes things load much faster.
- **GPU:** An NVIDIA graphics card is optional, but it helps if we ever decide to run AI models locally.

Software Requirements:

- **Operating System:** Windows 11, Ubuntu Linux, or macOS.
- **Programming Languages:** Python 3.10 or newer for the backend, and JavaScript for the frontend.
- **Mobile Tools:** Node.js, Expo CLI, and either Android Studio Emulator or iOS Simulator for testing the app.
- **Frameworks:** FastAPI and React Native.

- **External Connections:** We need API keys for the LLMs (like Gemini) to make the AI features work.

5.2 Project Scope and Constraints

For this phase of the Capstone project, we focused on delivering these main features:

- A system that generates personalized meal plans and calorie goals.
- A prompt setup that gives the AI the right context for accurate recipes.
- A validation module using Pydantic to make sure the data is structured correctly.
- A working mobile app dashboard that ties everything together.

Operational Constraints:

- **Latency:** Because we rely on external Gemini API calls, meal generation can take around 10 seconds before we get a response or trigger a fallback. This is slower than traditional apps but necessary for the complex AI logic.
- **Costs:** We have to monitor how many tokens the AI uses, so we try to keep our prompts as short and efficient as possible.
- **Security:** We have to be careful with user data, making sure personal health details are kept private and secure.

5.3 Sustainable Development Goal (SDG) Impact

We wanted this project to have a real positive impact, not just be a coding exercise. KEDO lines up well with **UN Sustainable Development Goal 3 (Good Health and Well-being) - Target 3.4**: *By 2030, reduce by one third premature mortality from non-communicable diseases through prevention and treatment.*

KEDO promotes preventive healthcare by helping people manage their diet better. By making this a free or accessible mobile app, we hope to give average users the kind of personalized nutrition advice that usually requires an expensive dietitian.

Chapter 6

IMPLEMENTATION SNAPSHOTS

The following screenshots show what the KEDO app actually looks like. The frontend, built with React Native, is designed to be simple and easy for anyone to use.

Figure 6.1: KEDO Application Loading & Initial Dashboard View. This screen welcomes the user and gets the backend ready to load personalized recommendations.

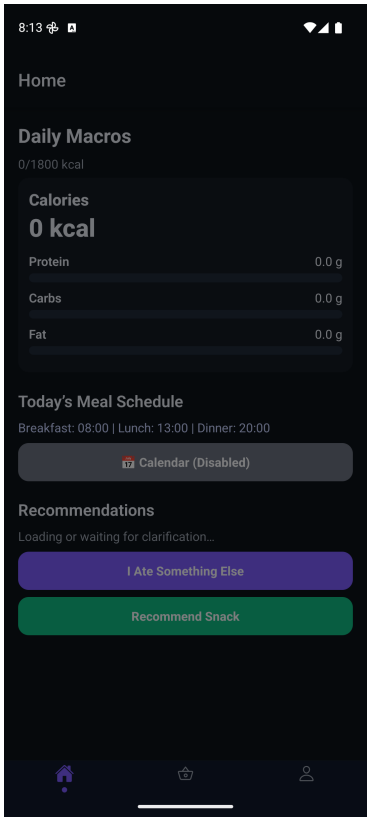

Figure 6.2: Interactive Dashboard with AI Recommendations. This view shows the app successfully delivering meal ideas, calorie breakdowns, and daily tracking based on the user's profile.

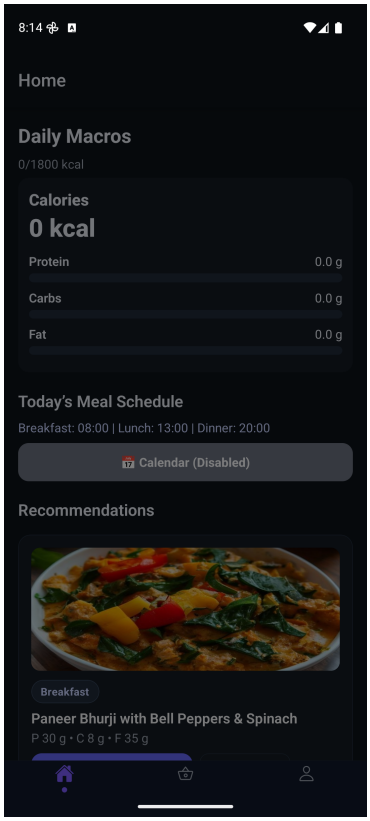

Figure 6.3: Detailed Meal and Nutritional Breakdown. Users can click on specific recipes to see exact ingredients, cooking steps, and nutritional facts that the AI generated.

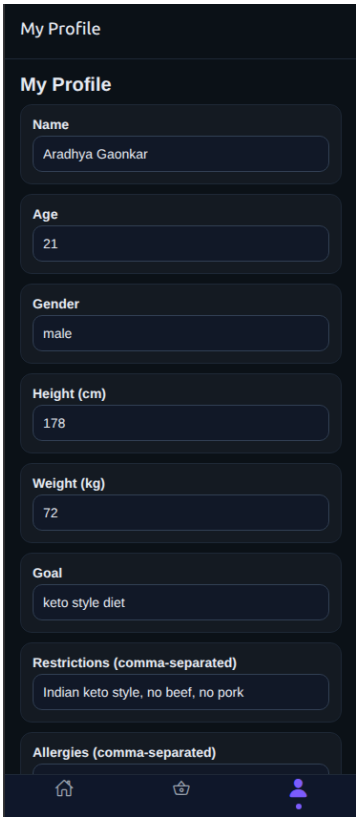

Figure 6.4: User Details & Health Configuration. This section collects the user's health profile data. It stores details like diet goals, allergies (like "no beef"), activity levels, and meal times. This information is sent directly to the AI to make sure the meal plans are safe and accurate.

Chapter 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The KEDO AI-Powered Personalized Nutrition Assistant represents a monumental and significant leap forward in the realm of preventative digital healthcare. By successfully fusing modern mobile engineering (React Native) with a high-performance backend (FastAPI) and state-of-the-art cognitive logic (Gemini LLMs), the platform successfully abstracts the complexity of nutritional science into an accessible, daily-use tool.

Crucially, KEDO solves the most dangerous problem currently facing Medical AI: Hallucinations. By keeping the architecture simple and functioning as a highly optimized API wrapper, it relies on strict Pydantic structural parsers and rigorous prompt constraints rather than over-engineered RAG pipelines. KEDO ensures that no medically unsafe or contradictory information reaches the user by explicitly feeding the LLM all hard constraints in a single, grounded payload.

KEDO ultimately embodies a reliable, highly context-aware bridge between artificial intelligence and human physical wellbeing, proving that LLMs can be deployed safely in health-adjacent sectors if properly constrained. In doing so, it successfully contributes to the reduction of non-communicable diseases as mandated by UN SDG 3.4.

7.2 Future Scope

Moving forward, the foundational architecture of KEDO is highly extensible. The future scope of this project includes:

- **Direct IoT Wearable Integration:** Absorbing real-time biometric data (heart rate variability, blood glucose levels) directly from smartwatches and continuous glucose mon-

itors to offer proactive, minute-by-minute dietary adjustments.

- **Image Recognition (Computer Vision):** Allowing users to take a photo of their meal, leveraging Vision models to autonomously estimate caloric intake and map it against their daily goals.
- **Integration with Medical Providers:** Creating a secure, compliant portal for dietitians and physicians to monitor their patients' progress and directly inject clinical constraints into the AI's generation context.

REFERENCES

[1] Ricci et al. Food recommendation system using content-based and collaborative filtering. *International Research Journal of Innovations in Engineering and Technology (IRJIET)*, 2024.

[2] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive tasks. In *Advances in Neural Information Processing Systems*, 2020.

[3] G. Wegener. Sort-ai: A structural safety and reliability framework for advanced ai systems with retrieval-augmented generation as a diagnostic testbed. *Preprints.org*, 2024.

[4] Shrestha et al. Integrating ai-driven predictive analytics in wearable iot for real-time health monitoring in smart healthcare systems. *MDPI Applied Sciences*, 2023.

[5] Gao et al. Mega-rag: A retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of llms in public health. *Frontiers in Public Health*, 2024.

[6] Meta Platforms Inc. React native - a framework for building native apps using react, 2025. Reference for cross-platform mobile frontend engineering.

[7] Tiangolo. Fastapi - high performance, easy to learn, fast to code, 2025. Reference for designing the asynchronous backend architecture.